used to indicate when a CPU's state changed; KOBJ_MOVE is used only by network interfaces when renaming a *kobject*.

You can find the kernel uevent framework implemented in *lib/kobject_uevent.c*. The uevent framework exports APIs *kobject_uevent and kobject_uevent_env*, which implement the user-space communication. The difference between the two APIs is that the latter allows a driver to send extra information to user space. This extra information is termed 'environmental data' by the framework. Internal to the framework, the *kobject_uevent* API is just a call of the *kobject_uevent_env* API with 'environmental data' set to NULL.

The basic message passed to user space from the kernel uevent framework is as follows:

```
ACTION=
DEVPATH=
SUBSYSTEM=
SEQNUM=
```

The *DEVPATH* string holds the path of the *kobject* and *SUBSYSTEM* indicates the subsystem the uevent originated from. Any extra data other than the above four, is termed as environment data. Information in this environment data is specific to the subsystem or the kernel object. The environment data, which is basically an array of strings, holds state changes or other information useful to user-space code. The environment data is packed in a *kobj_uevent_env* data structure, defined in */include/linux/kobject.h*:

```
struct kobj_uevent_env {
        char *envp[UEVENT_NUM_ENVP];
        int envp_idx;
        char buf[UEVENT_BUFFER_SIZE];
        int buflen;
};
```

The uevent framework provides the *add_uevent_var* API to add more environment data to a uevent message.

How it is used by a driver

Let us now see how a driver uses this framework to send information to user space, with an example—a simple scenario of an Android device connected to a host PC as a USB device. When an Android device is connected as a USB device, the Android framework notifies the user with a USB icon in the notification bar. To do this, the Android USB framework needs certain information from the kernel when the Android device is configured as a USB device. This information is passed on through uevents by the kernel USB driver. Let us see how USB-specific state information is passed on to the Android USB framework.

The following code is from the *android_work* function of the *drivers/usb/gadget/android.c* file; you can browse the code at *https://android.googlesource.com/*. The USB gadget

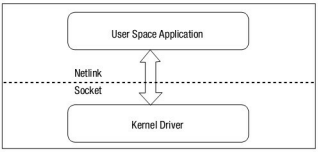

Figure 1: A simple representation of the uevent mechanism

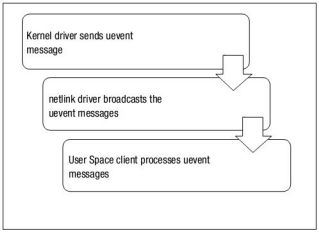

Figure 2: The flow of uevents from kernel to user space

driver forms three different state strings with the keyword USB_STATE, in a format similar to *kobj_uevent_env* environment data, as shown below:

```
char *disconnected[2] = { "USB_STATE=DISCONNECTED", NULL };
        char *connected[2] = { "USB_STATE=CONNECTED", NULL };
 char *configured[2] = { "USB_STATE=CONFIGURED", NULL };
 char **uevent_envp = NULL;
```

When an Android device is connected as a USB device, the state of the device is checked from the gadget driver's flags and appropriate environment data is assigned to the *uevent_envp* variable. For example, when the device is connected to the PC, *USB_STATE=CONNECTED* is set and when the drivers are installed successfully and the device is functional, *USB_STATE=CONFIGURED* is set.

```
 if (cdev->config)
    uevent_envp = configured;
else if (dev->connected != dev->sw_connected)
    uevent_envp = dev->connected ? connected : disconnected;
```

This state information is then propagated to the user space using *kobject_uevent_env* with the action *KOBJ_CHANGE*, as shown below:

```
 if (uevent_envp) {
```